Improved Security Controls for Shared Source Code Repositories

Enhancing protection in collaborative software development

Security Controls in Shared Source Code Repositories

Overview of Security Controls for Shared Repositories

Recognizing Security Boundaries

View repositories as critical security boundaries, not just code storage, to prevent unauthorized changes and exposure risks.

Governance and Access Controls

Implement governance frameworks, multi-factor authentication, and role-based permissions to ensure least-privilege access.

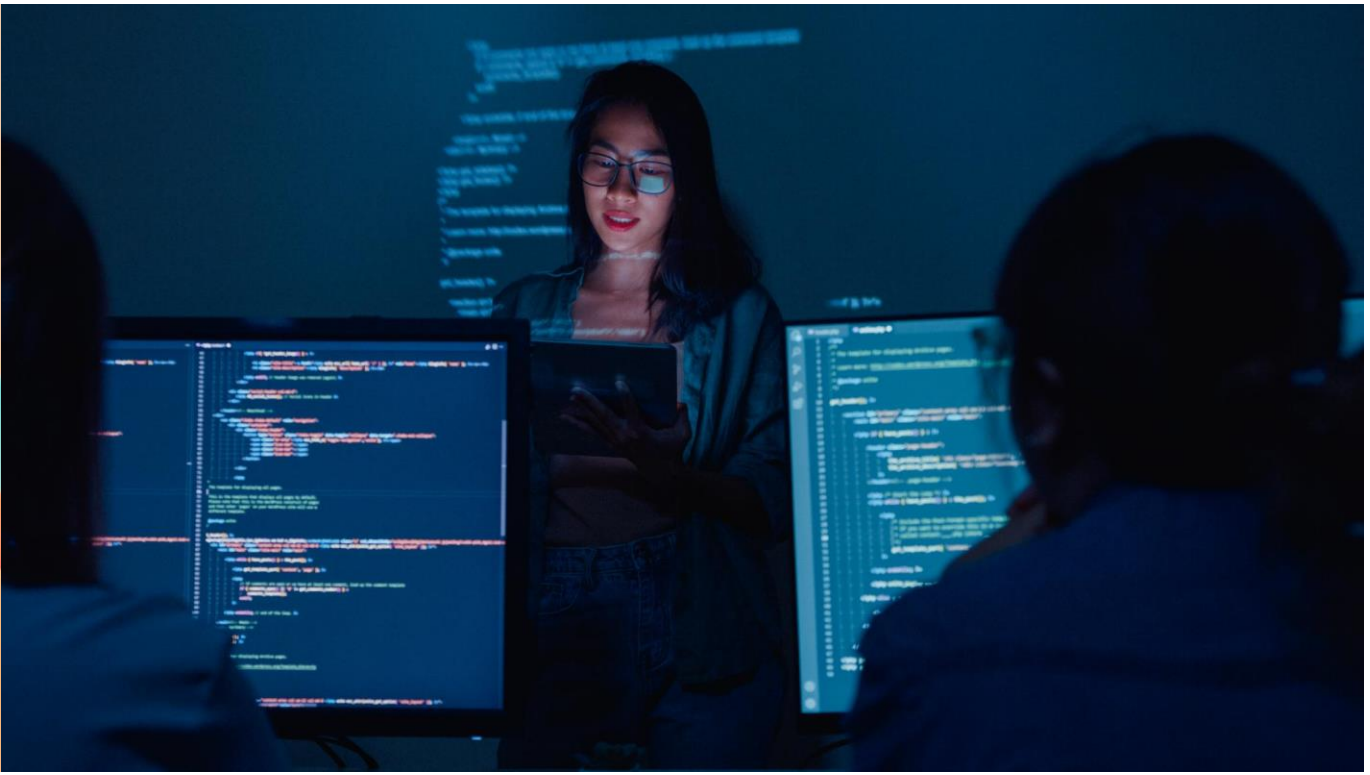
Automated Security Scanning

Use automated scanning for secrets, dependencies, linting, and static analysis to detect risks early in the development workflow.

Enforcing Secure Collaboration

Enable branch protection, CODEOWNERS rules, and audit logs to maintain code integrity and accountability across contributors.

Repository Risk Management



Importance of Strong Controls in Shared Repositories

Risks in Shared Repositories

Shared repositories face risks like direct commits, secret leaks, vulnerable dependencies, and CI/CD pipeline attacks.

Mitigating Risks with Controls

Strong controls enforce pull requests, limit branch modifications, and require status checks to reduce errors.

Security Through Automation

Automated scanning tools detect vulnerabilities early, ensuring a secure and auditable development process.

Identity and Access Controls

Implementing Least-Privilege Access and Strong Identity Protections

Centralized Authentication with MFA

Require contributors to authenticate through a centralized identity provider using mandatory multi-factor authentication to reduce unauthorized access risks.

Role-Based Access Control

Enforce RBAC so users have only minimum permissions needed, using group-based controls to simplify audits and access reviews.

Separation of Duties

Separate roles such as administrators and reviewers to prevent single points of failure and reduce misuse opportunities.

Regular Access Reviews

Conduct frequent reviews of access rights after personnel or team changes to maintain least-privilege principles.

Branch Protections

Safeguarding Main and Release Branches

Branch Protection Basics

Protecting main and release branches prevents unauthorized changes and ensures code integrity.

Pull Request Enforcement

Requiring pull requests ensures all changes undergo review, testing, and validation before merging.

Mandatory Status Checks

Status checks verify build success, tests, and security scans before changes are integrated.

Audit and Stability

Protecting branches maintains production stability and supports auditing with clear change histories.

Code Review and Ownership

Strengthening Review Processes Through CODEOWNERS

Peer Review for Quality

Requiring one to two approving reviews before merging ensures peer oversight and fosters accountability in the codebase.

Role of CODEOWNERS

CODEOWNERS designate experts for specific code areas, ensuring knowledgeable review of sensitive or high-risk components.

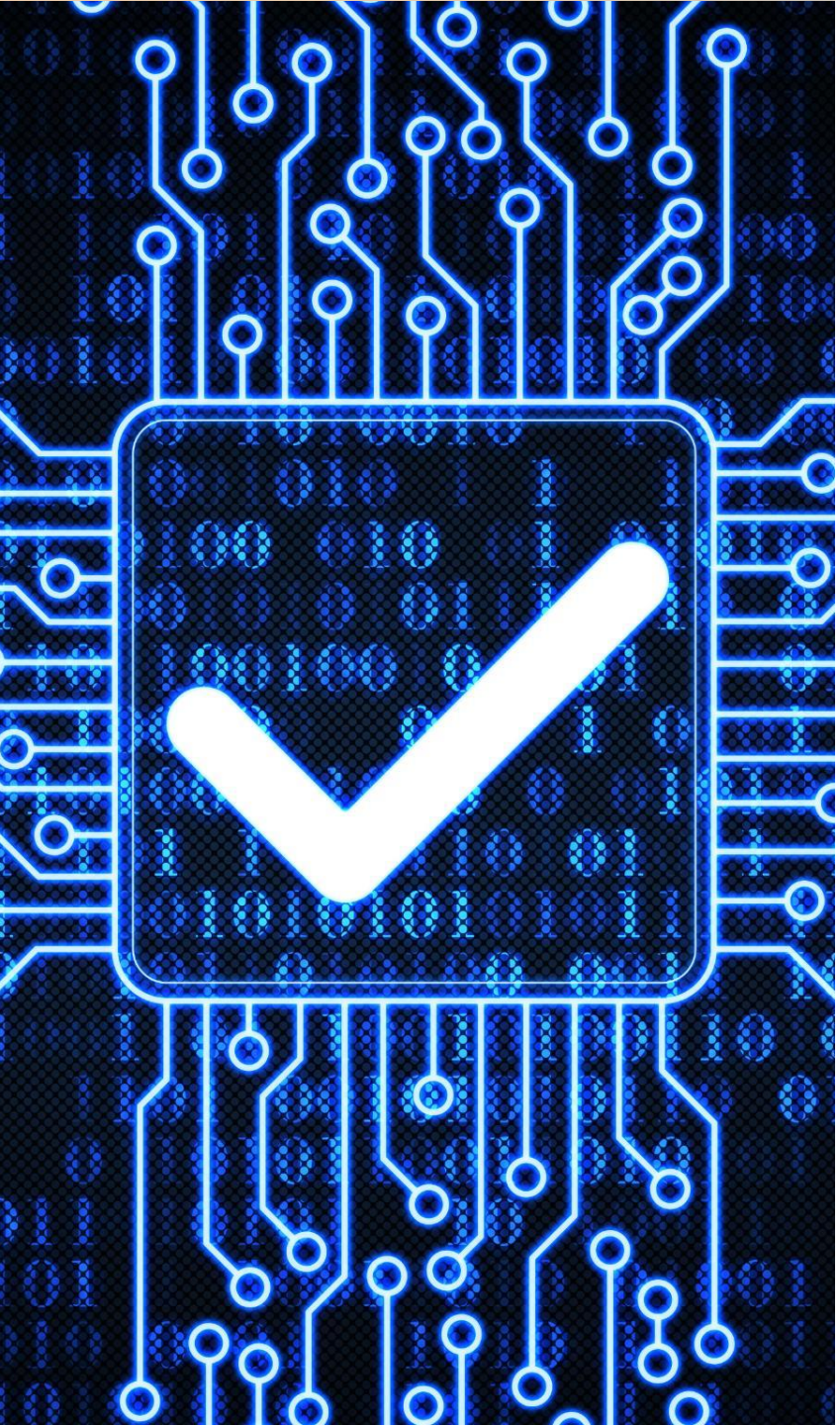
Security and Merge Controls

Restricting review dismissals and limiting merge permissions prevents unauthorized changes and enhances security.

Improved Code Quality

These review practices improve architectural consistency, readability, and enable early detection of security issues.

Push-Time Guardrails



Implementing Commit-Level Safeguards

Push-Time Commit Validation

Push rules block commits with sensitive data or non-compliant messages before entering the repository.

Cryptographically Signed Commits

Signed commits enhance provenance tracking and prevent identity forgery and malicious changes.

Linear History Enforcement

Enforcing linear commit history simplifies audits by ensuring predictable and traceable changes.

Securing CI Workflow Files

Restricting access to CI workflow files prevents supply chain attacks and unauthorized automation changes.

Automated Repository Checks

Enhancing Security With Automated Scanning

Secrets Scanning

Secrets scanning tools detect exposed credentials early to prevent unauthorized access to sensitive systems.

Dependency Scanning

Dependency scanning identifies vulnerable libraries and alerts developers to insecure or outdated packages.

Static Application Security Testing

SAST tools analyze source code to find vulnerabilities like injection flaws and insecure configurations.

Standardization and Automation

Linting, style enforcement, PR templates, and checklists standardize reviews and reduce defects and overhead.

Securing CI/CD Pipelines

Protecting Build and Deployment Workflows

Least-Privilege Access

Use short-lived, scoped credentials for automation tokens to limit risk if compromised.

Third-Party Action Verification

Verify and pin third-party plugins to reduce risk of malicious dependencies in build workflows.

Restricted Pipeline Modifications

Limit who can modify pipeline definitions to prevent unauthorized changes and security risks.

Deployment Approvals and Monitoring

Add explicit deployment approvals and log permission changes for incident response support.

Integrity and Monitoring

Provenance, Auditing, and Tamper Detection

Build Provenance Verification

Verify build origins to ensure artifacts come from trusted, authenticated source code and hardened environments.

Audit Log Monitoring

Audit logs capture repository actions like permission changes and security events to detect anomalies swiftly.

Incident Response Playbooks

Well-documented procedures guide revoking tokens, rotating secrets, and quarantining suspicious branches effectively.

Continuous Monitoring & Alerts

Automated monitoring and alerts provide visibility to prevent or mitigate tampering in software repositories.